

Operating System (BCSE303P) · Course Project – Review 3

Adaptive CPU Scheduling Using Predicted Burst Lengths

Evaluation, robustness to prediction errors, comparison and conclusion

Shivaansh Aggarwal (24BCE0883) · Devansh Sharma (24BCE0717)

Slot L15+L16 · Faculty: Dr. Anisha M. Lal

The problem and our approach

The problem

- SJF is optimal but needs the next burst length, which the OS does not know
- Round Robin's quantum is fixed by hand and never revisited
- Prediction-based schedulers trust the prediction: a burst that outruns it keeps looking short
- With time slicing, such a burst returns after every quantum and is picked again

1

Who runs next

P = predicted time left − β × time waited. Lowest P runs; ageing (β = 0.03) prevents starvation.

2

How long it runs

Quantum = 80th percentile of predictions × recent error ratios, updated every 50 decisions (1–100 ms).

3

Overrun guard (new)

Once a burst passes its prediction × median error ratio, time left = max(prediction, 0.25 × CPU used).

How we evaluated it

7
workloads (W1–W6 + adversarial W7)

14
scheduler configurations

10
evaluation seeds, all runs paired

53 / 53
automated tests pass

Seeds kept apart

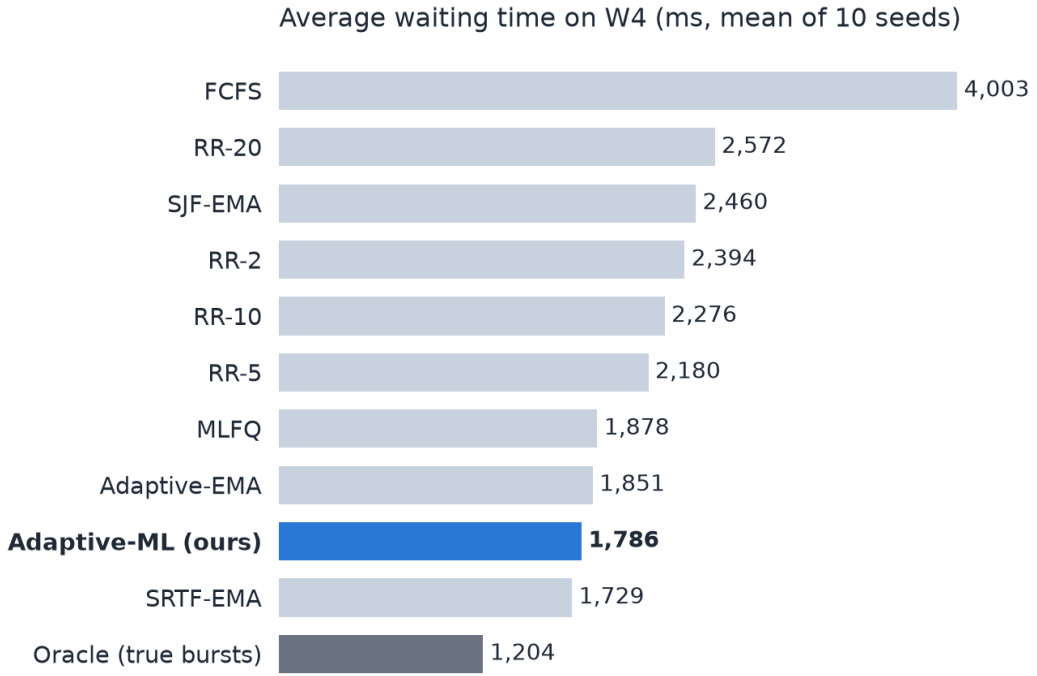
- Training the predictor: seeds 1000–1004
- Tuning β and the guard: seeds 100–102
- Evaluation: seeds 0–9 (never used for tuning)

Statistics

- Paired bootstrap 95% confidence intervals
- Exact Wilcoxon signed-rank test, Holm-corrected
- Significant only if Holm $p < 0.05$; losses reported too

W7 is intentionally adversarial: 30% bait processes (many ~1 ms bursts, then one ~400 ms burst) that make up 76% of the CPU demand. The simulator reproduces the textbook answers exactly (FCFS 17, SJF 3, RR 5.67, SRTF 6.5 ms).

Mixed workload: 27% less waiting than SJF



−27.4%

avg wait vs SJF-EMA, 95% CI −32.4 to −23.7%, better on 10 of 10 seeds

54%

of the gap to the oracle (SRTF with true bursts) closed

−61%

context switches vs RR-5, response 15 ms vs 23 ms

Across the six standard workloads

Workload	Avg wait vs SJF-EMA	Worst wait vs SJF-EMA	Seeds better	Avg significant?
W1 CPU-bound	−1.9%	−56% *	7 / 10	No
W2 I/O-bound	+13.8%	+0.5%	0 / 10	Yes (loss)
W3 Interactive	+5.3%	−27% *	2 / 10	No
W4 Mixed	**−27.4%**	**−23% ***	**10 / 10**	**Yes**
W5 Phase-changing	−1.7%	−53% *	4 / 10	No
W6 Overload	+2.6%	−2%	2 / 10	No

** worst-case change significant after Holm correction ($p < 0.05$)*

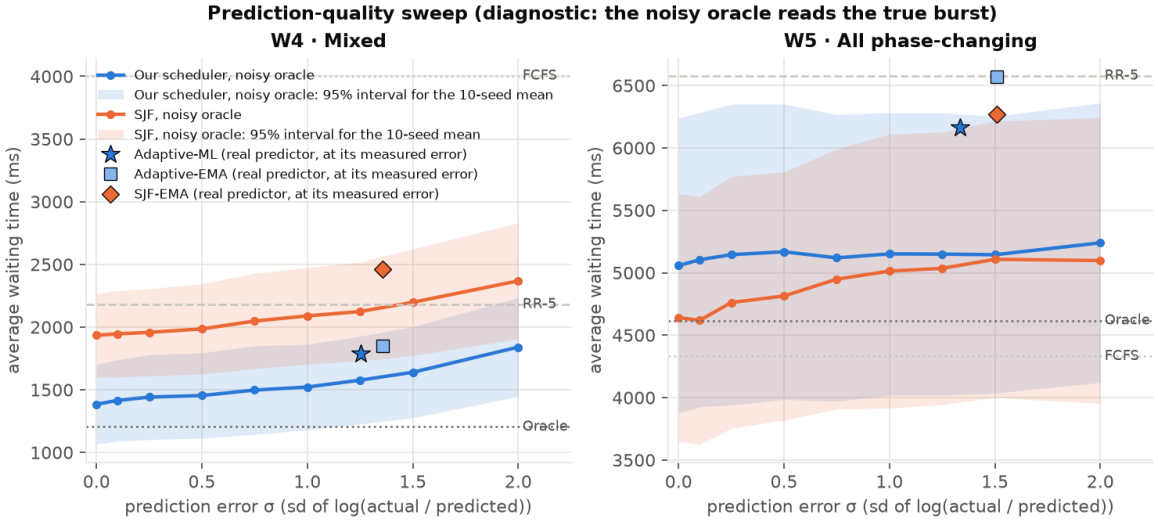
Why it works

The quantum follows the workload (≈6 ms interactive, ≈97 ms CPU-bound, 3.4 ms I/O). Ageing halves the worst case. Prediction alone gains only 5%.

Where it does not

All-I/O work is 13.8% worse (significant): tiny random bursts leave nothing to predict. Fairness index slightly below SJF-EMA on 5 of 6.

How much does prediction quality matter?



Diagnostic only: the noisy oracle reads the true burst and adds error exp(σZ)

On W5 the curve is flat (5,057 → 5,240 ms): there the time slicing, not the predictor, is the bottleneck.

−28.4%

W4: our mechanism vs SJF, both with perfect predictions (10/10)

22.5%

W4: further gain possible with perfect predictions

σ 1.25 → 1.86

real errors cost more than random errors of the same size

The weakness: a long burst that looks short

1. **Predicted 1.2 ms**
   A bait process has run 15 tiny bursts, so the model expects another tiny one.

2. **Runs 40 ms, still looks like 1.2 ms**
   Without the guard it returns after each quantum with a 1.2 ms estimate and is picked again.

3. **Guard: max(1.2, 0.25 × 40) = 10 ms**
   A new interactive burst predicted at 5 ms now runs first; after 200 ms the bait burst looks like 50 ms.

W7 bait-and-switch

- Intentionally adversarial
- 50% interactive, 20% I/O-bound, 30% bait
- Bait: 10–20 bursts of ~1 ms, then one ~400 ms burst, twice
- Bait processes: 76% of the CPU demand
- Guard tuned on seeds 100–102, tested on 0–9

Live demo: the guard on W7

```
ai_cpu_scheduler — -zsh — 125×26

(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler % python demo_innovation.py
W7 · Bait-and-switch (50% interactive, 20% I/O, 30% bait), seed 0
200 processes, 66 bait. Example bait process (ms): 1.8, 1.4, 1.4, 1.7, 0.1, 1.2, ..., 464 (!), ...

Scheduler        avg wait  interactive wait  interactive resp.  bait wait  worst wait
------------------------------------------------------------------------------------
SJF-EMA             2,570             3,065              258.9      2,237      10,042
RR-5                  978               448                9.9      2,089       6,232
MLFQ                  826               203                0.9      2,173       6,698
Adaptive-ML         1,477             1,221               91.8      2,544       8,874
Adaptive-ML-G         984               103                4.7      2,777      12,875

All times in ms.  Adaptive-ML-G = Adaptive-ML + overrun guard
Guard: once a burst runs past estimate x r50 (median observed actual/predicted),
its time left is taken as max(estimate, 0.25 x CPU time already used).
(.venv) Shivaansh@Shivaanshs-MacBook-Air ai_cpu_scheduler %
```

Below: 10-seed means, paired tests

Real Terminal output of python demo_innovation.py (W7, seed 0)

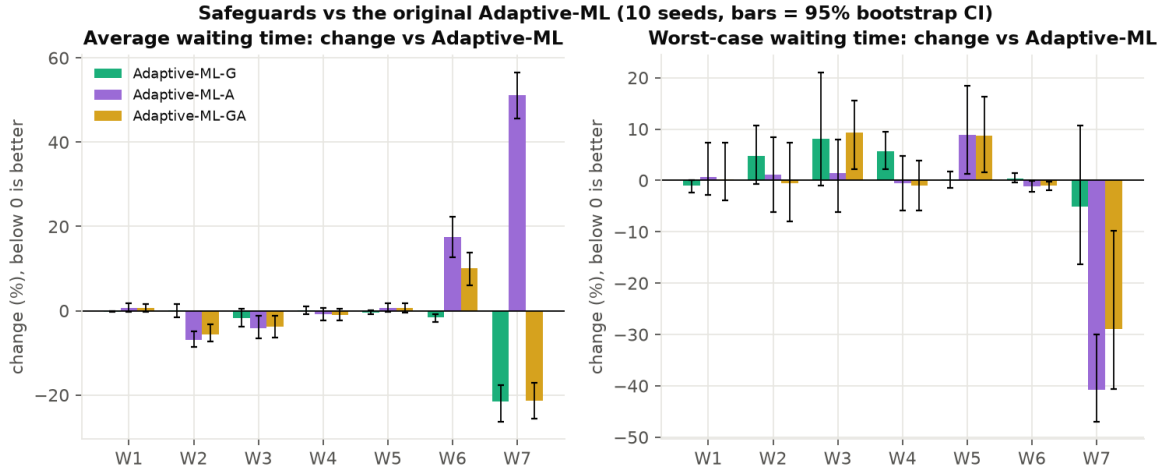
−21.5%
avg wait vs Adaptive-ML (10/10)

−92%
interactive wait (1,399 → 112 ms)

+9.3%
MLFQ still has the lower avg wait

≈2×
worst-case wait of RR-5 and MLFQ

Trade-offs and a negative result



Change vs Adaptive-ML (green: guard, purple: persistent ageing, gold: both); bars = 95% CI

Recommended configuration: Adaptive-ML with the overrun guard.

Guard elsewhere

W1–W5 average: −1.8% to +0.1%, none significant. W4 worst case +5.6% (not significant after correction).

Persistent ageing

Fixes a real mismatch (the reset discards 70% of W4 waiting) but avg wait +17.5% on W6, +51% on W7. Not retained.

Comparison with existing schedulers

Scheduler	W4 avg (ms)	W4 worst (ms)	W4 switches	W7 avg (ms)
FCFS	4,003	19,407	3,962	3,630
SJF-EMA	2,460	23,766	3,977	3,847
SRTF-EMA	1,729	24,903	6,902	4,045
RR-5	2,180	14,887	15,328	1,741
MLFQ	1,878	15,016	11,888	1,495
Adaptive-ML (ours)	**1,786**	**18,388**	**5,933**	**2,083**
Adaptive-ML-G (ours + guard)	**1,786**	**19,426**	**5,938**	**1,634**

Mean of 10 evaluation seeds.

- W4: lowest average wait except SRTF (3% better, not significant; 35% higher worst case)
- W4: MLFQ close on average but needs twice the switches
- W7: MLFQ has the lowest average; the guard beats RR-5 and gives interactive work the shortest waits
- Unlike dynamic-quantum papers, burst lengths are never assumed known

Conclusion and future work

Conclusion

- Using the prediction twice gives 27% less waiting on mixed work
- The gain comes mainly from the mechanism, not the model
- The overrun guard closes a real weakness: −92% interactive wait on W7
- Not a complete defence; worse on all-I/O work; persistent ageing not kept

Future work

- Real program traces and other adversarial patterns
- A worst-case bound for the guard
- Load-aware ageing
- A small tree or lookup table inside xv6

Thank you